

Doc. no.: P0219R1  
Date: 2016-06-24  
Beman Dawes <bdawes at acm dot org>  
Reply to: Jamie Allsop <jamie dot allsop at gmail dot com>  
Nicolai Josuttis <nico at josuttis dot de>  
Audience: Library, Filesystem

# Relative Paths for Filesystem (R1)

## [Introduction](#)

### [History](#)

### [Preliminary implementation](#)

## [Requirements](#)

## [Issues](#)

## [Design decisions](#)

[Provide separate lexical and operational relative functions](#)

[Provide separate lexical and operational proximate functions](#)

[Add lexical functions as path member functions](#)

[Provide a lexically normal member function returning a normal form path](#)

[Provide a weakly canonical operational function](#)

[Resolve issues in ways that "just work" for users](#)

[Specify lexical relative in terms of mismatch](#)

[Specify operational relative in terms of weakly canonical](#)

[Specify operational relative in terms of lexically relative](#)

[Propose common prefix and remove common prefix functions separately, as algorithms](#)

## [Proposed wording](#)

[Define \*normal form\*](#)

[New class path member functions](#)

[Synopsis](#)

[Specification](#)

[New operational functions](#)

[Synopsis](#)

[Specification](#)

[LWG guidance requested](#)

[References](#)

## Introduction

The [Boost Filesystem library](#) has had user requests for a relative path function for more than ten years. For the [File System TS](#), the issue was raised as LWG 2611, PDTs comment GB-1, and resolved as *NAD Future*. A proposed solution, [P0011R0, Additions to Filesystem supporting Relative Paths](#), is currently before the committee.

The requested functionality seems simple - given two paths with a common prefix, return the non-common suffix portion of one of the paths such that it is relative to the other path.

In terms of the Filesystem library,

```
path p("/a/b/c");
path base("/a/b");
path rel = relative(p, base); // the requested function
cout << rel << endl;        // outputs "c"
assert(absolute(rel, base) == p);
```

If that was all there was to it, Filesystem would have had a relative function years ago. Other libraries supply such a function, such as [Python's os.path.relpath](#), albeit with simplistic semantics.

The problem with simplistic semantics:

- Clashing requirements
- Symlinks
- Directory placeholders (*dot*, *dot-dot*)
- User-expectations
- Corner cases.

In previous discussions, committee members have made it clear that they do not want an overly simplistic solution.

## History

[P0011R0, Additions to Filesystem supporting Relative Paths](#) by Jamie Allsop and Nico Josuttis is what broke the "relative" logjam. Much of what follows is based directly on that proposal. The

weakly\_canonical function and details of the semantic specifications such as use of mismatch were introduced by Beman Dawes.

## Implementation experience

[Boost Filesystem 1.60.0](#) shipped a preliminary implementation of this proposal in December, 2015.

## Requirements

**Requirement 1:** Some uses require symlinks be followed; i.e. the path must be resolved in the actual file system.

**Requirement 2:** Some uses require symlinks not be followed; i.e. the path must not be resolved in the actual file system.

**Requirement 3:** Some uses require removing redundant current directory (*dot*) or parent directory (*dot-dot*) placeholders.

**Requirement 4:** Some uses do not require removing redundant current directory (*dot*) or parent directory (*dot-dot*) placeholders since the path is known to be already in normal form.

## Issues

**Issue 1:** What happens if p and base are themselves relative?

**Issue 2:** What happens if there is no common prefix? Is this an error, the whole of p is relative to base, or something else?

**Issue 3:** What happens if p, base, or both are empty?

**Issue 4:** What happens if p and base are the same?

**Issue 5:** How is the "common prefix" determined?

**Issue 6:** What happens if portions of p or base exist but the entire path does not exist and yet symlinks need to be followed?

**Issue 7:** What happens when a symlink in the existing portion of a path is affected by a directory (*dot-dot*) placeholder in a later non-existent portion of the path?

**Issue 8:** Overly complex semantics (and thus specifications) in preliminary designs made reasoning about uses difficult.

**Issue 9:** Some uses never have redundant current directory (*dot*) or parent directory (*dot-dot*) placeholders, so a removal operation would be an unnecessary expense although otherwise harmless.

## Design decisions

### Provide separate lexical and operational relative functions

Resolves the conflict between [requirement 1](#) and [requirement 2](#) and ensures both requirements are met.

A purely lexical function is needed by users working with directory hierarchies that do not actually exist.

An operational function that queries the current file system for existence and follows symlinks is needed by users working with actual existing directory hierarchies.

### Provide separate lexical and operational proximate functions

Although not the only possibility, a likely fallback when the relative functions cannot find a relative path is to return the path being made relative. As a convenience, the proximate functions do just that.

### Add lexical functions as path member functions

This is the Filesystem library convention, but it is controversial. See [LWG guidance requested](#).

### Provide a [lexically normal](#) member function returning a [normal form](#) path

Enables resolution of [requirement 3](#) and [requirement 4](#) in a way consistent with [issue 9](#). Is a contributor to the resolution of [issue 8](#).

"Normalization" is the process of removing redundant current directory (*dot*), parent directory (*dot-dot*), and directory separator elements.

Normalization is a byproduct the current canonical function. But for the path returned by the proposed [weakly\\_canonical](#) function, only any leading canonic portion is in canonical form. So any trailing portion of the returned path has not been normalized.

Boost.filesystem has a non-const normalization function that was deprecated because it modifies the path, and that sometimes caused user confusion. We believe that a const function returning a path is a better solution.

### **Provide a [weakly\\_canonical](#) operational function**

Resolves [issue 6](#), [issue 7](#), [issue 9](#), and is a contributor to the resolution of [issue 8](#).

The operational function `weakly_canonical(p)` returns a path composed of `canonical(x)/y`, where `x` is a path composed of the longest leading sequence of elements in `p` that exist, and `y` is a path composed of the remaining trailing non-existent elements of `p` if any. "weakly" refers to weakened existence requirements compared to the existing canonical function.

- Having `weakly_canonical` as a separate function, and then specifying the processing of operational relative arguments in terms of calls to `weakly_canonical` makes it much easier to specify the operational relative function and reason about it. The difficulty of reasoning about operational relative semantics before the invention of `weakly_canonical` was what led to its initial development.
- Having `weakly_canonical` as a separate function also allows use in other contexts.
- Specifying the return be in [normal form](#) is an engineering trade-off to resolve [issue 7](#) in a way that just works for most use cases.
- Specifying normative encouragement to not perform unneeded normalization is a reasonable resolution for [issue 9](#).

### **Resolve issues in ways that "just work" for users**

Resolves issues [1](#), [2](#), [3](#), [4](#), [6](#), and [7](#). Is a contributor to the resolution of [issue 8](#).

The "just works" approach was suggested by Jamie Allsop. It is implemented by specifying a reasonable return value for all of the "What happens if..." corner case issues, rather than treating them as hard errors requiring an exception or error code.

### **Specify [lexically relative](#) in terms of `std::mismatch`**

Resolves [issue 5](#). Is a contributor to the resolution of [issue 8](#).

### **Specify operational [relative](#) in terms of [weakly\\_canonical](#)**

Is a contributor to the resolution of [issue 8](#).

- Covers a wide range of uses cases since a single function works for existing, non-existing, and partially existing paths.
- Works correctly for partially existing paths that contain symlinks.

### **Specify operational [relative](#) in terms of [lexically relative](#)**

Is a contributor to the resolution of [issue 5](#) and [issue 8](#).

It would be confusing to users and difficult to specify correctly if the two functions had differing semantics:

- When either or both paths are empty.
- When all elements of the two paths match exactly.
- Because different matching algorithms were used.
- Because although the same matching algorithm was used, it was applied in different ways.

These problems are avoided by specifying operational relative in terms of lexical relative after preparatory calls to operational functions.

### **Propose `common_prefix` and `remove_common_prefix` functions separately, as algorithms**

[P0011R0](#) proposed `common_prefix` and `remove_common_prefix` functions. In subsequent discussions between Allsop, Dawes, and Josuttis, it became clear that these functions are generic algorithms and are generally useful, but are not actually required for purposes of this filesystem proposal.

So these two functions are not a part of this proposal. Instead, a separate proposal will suggest adding these to clause 24, Algorithms library.

## Proposed wording

Add a new definition to 27.10.4 Terms and definitions [fs.definitions]:

### 27.10.4.n

#### normal form [fs.def.normal.form]

A path in **normal form** has no redundant current directory (*dot*) elements, no redundant parent directory (*dot-dot*) elements, and no redundant *directory-separators*. The normal form for an empty path is an empty path. The normal form for a path ending in a *directory-separator* that is not the root directory has a current directory (*dot*) element appended.

[Note: The rule that appends a current directory (*dot*) element supports operating systems like OpenVMS that use different syntax for directory names and regular-file names. —end note]

Add a new sub-section to the synopsis in 27.10.8 Class path [class.path] after // 27.10.8.4.10, query:

```
// [path.gen] Generation
path lexically_normal() const;
path lexically_relative(const path& base) const;
path lexically_proximate(const path& base) const;
```

Add a new sub-section after // 27.10.8.4.10 [path.query]:

#### 27.10.8.4.11 path generation [path.gen]

```
path lexically_normal() const;
```

Returns: \*this in [normal form](#).

[Example:

```
assert(path("foo/.bar/..").lexically_normal() == "foo");
assert(path("foo/./bar/..").lexically_normal() == "foo/.");
```

The above assertions will succeed. The second example ends with a current directory (*dot*) element appended to support operating systems that use different syntax for directory names and regular-file names.

On Windows, the returned path's *directory-separator* characters will be backslashes rather than slashes, but that does not affect path equality. —end example]

```
path lexically_relative(const path& base) const;
```

Returns \*this made relative to base. Does not resolve symlinks. Does not first normalize \*this or base.

Effects:

Using operator== to determine if elements match, determines the first mismatched element of \*this and base using mismatch(begin(), end(), base.begin(), base.end()).

Then:

- return path() if the first mismatched element of \*this is equal to begin() or the first mismatched element of base is equal to base.begin(), or
- return path(".") if the first mismatched element of \*this is equal to end() and the first mismatched element of base is equal to base.end(), or
- return an object of class path composed via application of operator/= (path("../")) for each element in the half-open range [first mismatched element of base, base.end()), and then application of operator/= for each element in the half-open range [first mismatched element of \*this, end()).

[Example:

```
assert(path("/a/d").lexically_relative("/a/b/c") == "../d");
assert(path("/a/b/c").lexically_relative("/a/d") == "../b/c");
assert(path("a/b/c").lexically_relative("a") == "b/c");
assert(path("a/b/c").lexically_relative("a/b/c/x/y") == "../");
```

```
assert(path("a/b/c").lexically_relative("a/b/c") == ".");
assert(path("a/b").lexically_relative("c/d") == "");
```

The above assertions will succeed. On Windows, the returned path's *directory-separators* will be backslashes rather than forward slashes, but that does not affect path equality. —end example]

[Note: If symlink following semantics are desired, use the operational function [relative\(\)](#) —end note]

[Note: If [normalization](#) is needed to ensure consistent matching of elements, apply [lexically\\_normal\(\)](#) to \*this, base, or both. —end note]

```
path lexically_proximate(const path& base) const;
```

*Returns:* If the value of `lexically_relative(base)` is not an empty path, return it. Otherwise return `*this`.

[Note: If symlink following semantics are desired, use the operational function [proximate\(\)](#) —end note]

[Note: If [normalization](#) is needed to ensure consistent matching of elements, apply [lexically\\_normal\(\)](#) to \*this, base, or both. —end note]

*Add the following new filesystem operational functions to 27.10.6 Header <filesystem> synopsis [fs.filesystem.syn]. Please maintain the alphabetic ordering of operational functions:*

```
...
path proximate(const path& p, error_code& ec);
path proximate(const path& p, const path& base=current_path());
path proximate(const path& p, const path& base, error_code& ec);
...
path relative(const path& p, error_code& ec);
path relative(const path& p, const path& base=current_path());
path relative(const path& p, const path& base, error_code& ec);
...
path weakly_canonical(const path& p);
path weakly_canonical(const path& p, error_code& ec);
...
```

*Add the following new filesystem operational functions to 27.10.15 Filesystem operation functions [fs.op.funcs]. Please maintain the alphabetic ordering of operational functions:*

#### **27.10.15.n Proximate[fs.op.proximate]**

```
path proximate(const path& p, error_code& ec);
```

*Returns:* `proximate(p, current_path(), ec)`.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
path proximate(const path& p, const path& base=current_path());
path proximate(const path& p, const path& base, error_code& ec);
```

*Returns:* For the first form, [weakly\\_canonical\(p\).lexically\\_proximate\(weakly\\_canonical\(base\)\)](#). For the second form, [weakly\\_canonical\(p, ec\).lexically\\_proximate\(weakly\\_canonical\(base, ec\)\)](#) except `path()` at the first error occurrence, if any.

*Throws:* As specified in Error reporting ([fs.err.report]).

#### **27.10.15.n Relative [fs.op.relative]**

```
path relative(const path& p, error_code& ec);
```

*Returns:* `relative(p, current_path(), ec)`.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
path relative(const path& p, const path& base=current_path());
path relative(const path& p, const path& base, error_code& ec);
```

*Returns:* `p` made relative to `base`. Resolves symlinks and normalizes both `p` and `base` before other processing.

*Returns:* For the first form, [weakly\\_canonical\(p\).lexically\\_relative\(weakly\\_canonical\(base\)\)](#). For the second

form, [weakly canonical](#)(p, ec).[lexically relative](#)([weakly canonical](#)(base, ec)) except path() at the first error occurrence, if any.

*Throws:* As specified in Error reporting ([fs.err.report]).

#### 27.10.15.n Weakly Canonical [fs.op.weakly\_canonical]

```
path weakly_canonical(const path& p);  
path weakly_canonical(const path& p, error_code& ec);
```

Returns p with symlinks resolved and the result normalized.

*Effects:* Using the status(p) or status(p, ec), respectively to determine existence, return a path composed by operator/= from the result of calling canonical() without a base argument and with a path argument composed of the leading elements of p that exist, if any, followed by the elements of p that do not exist, if any. For the first form, canonical() is called without an error\_code argument. For the second form, canonical() is called with ec as an error\_code argument, and return path() at the first error occurrence, if any.

*Postconditions:* The returned path is in [normal form](#).

*Remarks:* Implementations are encouraged to avoid unnecessary normalization such as when canonical() has already been called on the entirety of p.

*Throws:* As specified in Error reporting ([fs.err.report]).

-- end proposed wording --

---

## LWG guidance requested

The Filesystem library is unusual in that it has several functions with both lexical (i.e. cheap) and operational (i.e. expensive due to file system access) forms with differing semantics. It is important that users choose the form that meets their application's specific needs. The library has always made the distinction via the convention of lexical functions being members of class path, while operational functions are non-member functions. The current wording of this proposal follows the path member function convention for the new lexical function. The lexical functions proposed here also use the name prefix `lexically_` to drive home the distinction.

For the contrary argument, see Sutter and Alexandrescu, *C++ Coding Standards*, 44: "Prefer writing nonmember nonfriend functions", and Meyers, *Effective C++ Third Edition*, 23: "Prefer non-member non-friend functions to member functions."

*At the Oulu meeting the LWG indicated a strong preference for the member / non-member convention as proposed and the naming as proposed.*

## References

[1] Beman Dawes, [N4100, Programming Languages — C++ — File System Technical Specification](#), 2014.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4100.pdf>

[2] Beman Dawes, others, [Boost Filesystem Library, V3](#), 2015.

[https://www.boost.org/doc/libs/1\\_60\\_0/libs/filesystem/doc/index.htm](https://www.boost.org/doc/libs/1_60_0/libs/filesystem/doc/index.htm)

[3] Jamie Allsop, Nicolai Josuttis, [P0011R0, Additions to Filesystem supporting Relative Paths](#), 2015.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0011r0.html>

[4] Python.org, [Function os.path.relpath documentation](#),

<https://docs.python.org/2/library/os.path.html#os.path.relpath>

---